Training Large Language Models

NECKER

July 15, 2026

Contents

1 Introduction: The Core Logic of LLMs

Large Language Models (LLMs) **predict the next piece of text** based on the context they have just processed. In essence, it is an extremely powerful and complex version of the autocomplete feature found on our smartphones.
This mechanism is known as *autoregressive generation*: the model analyzes a text, calculates the probabilities for the word that should follow, selects one, "pastes" it to the original text, and starts the process over again. It is a continuous cycle of reading and predicting.

1.1 Tokens

To make this mechanism work efficiently, models do not read sentences letter by letter, nor word by word. They use a unit of measurement called a **token**.

Imagine tokens as Lego bricks of variable sizes that compose language. The need to use them arises from two fundamental practical problems:

- **Why not letter by letter?** If the model read character by character ("t-r-e-e"), each individual unit would carry no semantic meaning on its own. Furthermore, to read a short paragraph it would have to process thousands of letters. Since the underlying architecture (the Transformer) struggles significantly (requiring quadratic computational power) as the text gets longer, using individual letters would make the model extremely slow and incapable of remembering long-term context.
- **Why not word by word?** If we used entire words, the model's vocabulary would have to contain every single existing word: all conjugated verbs, plurals, foreign words, neologisms, and typos. We would end up with a dictionary of millions of entries. Such a table would require an unsustainable amount of memory (VRAM) for any current hardware.

The perfect compromise: Tokens (or *sub-words*) solve both problems. Very common words (such as "house" or "cat") correspond to a single token. Rare or long words are instead "split" into multiple tokens. For example, the word "psychoneuroendocrinology" might be divided into the tokens "psycho", "neuro", "endocrino", "logy". In this way, with a fixed and compact dictionary (usually between 30,000 and 100,000 tokens), the model can compose and understand any text, assembling known fragments to handle words it has never seen before.

1.2 Epochs

An **epoch** is defined as one complete pass of the entire training dataset through the model. It means showing the model every single piece of data exactly once to allow it to update its weights. Passing the same dataset multiple times (multiple epochs) serves to "reinforce" knowledge, but there is a limit: overdoing it leads to overfitting (the model memorizes the data by heart instead of understanding the underlying logic).

1.3 Embeddings

Tokens themselves are merely integer numbers (IDs) pointing to a row in a vocabulary dictionary, but a plain number carries no semantic meaning for a neural network. To solve this problem, every token is converted into an **Embedding**.

An embedding is a dense vector of real numbers (often composed of thousands of dimensions, e.g., $d = 4096$). The revolutionary aspect of embeddings is that they translate semantics into geometry: words with similar meanings (such as "king" and "queen") will have vectors positioned very close to each other in this high-dimensional multidimensional space. In this manner, the model does not process words, but performs actual algebraic operations on concepts.

1.4 Parameters (The Network Weights)

When referring to "7 billion" or "70 billion" parameter models (such as Llama 3 or GPT-4), one is talking about the **weights** and *biases* of the matrices inside the neural network. Parameters are the "memory nodes" of the model. Initially, they are set to random values. During training, every time the model tries to guess the next word and makes a mistake, a mathematical algorithm (*Backpropagation*) slightly adjusts these billions of numbers to reduce the error on the next attempt. The intelligence of an LLM resides entirely within the final configuration of these immense matrices of real numbers.

1.5 Context Window

The autoregressive mechanism does not possess infinite memory. The model can only "look back" at a maximum number of tokens at a time: this limit is the **Context Window**. If a model has a context window of 8,000 tokens (approximately 6,000 words), once that limit is exceeded it will begin to "forget" the beginning of the conversation or document. From a computational standpoint, doubling the context window requires significantly more RAM (VRAM) and compute power (often with a quadratic cost scaling), which is why optimizing this parameter represents one of the most complex engineering challenges.

1.6 Training Phases: Pre-training and Fine-Tuning

The training of a modern LLM does not occur in a single monolithic block, but is divided into two crucial phases:

- **Pre-training (Base Training):** This is the most massive and expensive phase. The model "reads" terabytes of raw data from the internet (books, articles, source code) over several months, learning grammar, logic, programming, and real-world facts. In this phase, it simply learns to predict the next word in an unstructured manner.
- **Fine-Tuning (Alignment):** A pre-trained model does not know how to answer questions; it only knows how to complete sentences. Fine-Tuning (often performed using techniques like *Supervised Fine-Tuning* or reinforcement learning) utilizes much smaller, curated datasets to teach the model how to act as a helpful assistant, follow instructions (prompts), and respond in a conversational chat format.

2 The Multi-Stage Training Process

Training a state-of-the-art LLM is structured into a strictly sequential pipeline aimed at transforming a statistical string approximator into a fully-fledged assistant.

2.1 Pre-training (Self-Supervised Pre-training)

Pre-training represents the most computationally intensive phase (often absorbing over 98% of the total compute budget, measured in $10^{21} \sim 10^{24}$ FLOPs (number of **Floating Point Operations**)). The objective is to force the network to learn syntax, semantics, basic logical reasoning, and encyclopedic knowledge through simple next-token prediction.

1. Data Curation and Computational Flow:
The model is exposed to colossal datasets (on the order of trillions of tokens) composed of heterogeneous web fragments (e.g., Common Crawl), books, scientific papers, and source code. Before training, this data undergoes rigorous heuristic filtering (deduplication, removal of artifacts and low-quality content) to prevent the model from memorizing noise. During the *forward pass*, given an input sequence of length T, $X = (x_1, \ldots, x_T)$, the model produces for each position i a vector of *logits* $\mathbf{z}_i \in \mathbb{R}^{|\mathcal{V}|}$, where $|\mathcal{V}|$ is the vocabulary size. The predicted probability for the next token is obtained by applying the softmax function:

$$P_\theta(x_i \mid x_1, \ldots, x_{i-1}) = \frac{\exp(z_{x_i})}{\sum_{k=1}^{|\mathcal{V}|} \exp(z_k)} \tag{1}$$

Practical Example: Logits and Softmax Calculation

To understand the model's behavior during the *forward pass*, let us analyze a simplified scenario.

2.1.1 Scenario Setup

- **Vocabulary ($\mathcal{V}$):** The model knows only 4 words: $\mathcal{V} = \{\text{"cat"}, \text{"house"}, \text{"eats"}, \text{"mouse"}\}$. The vocabulary size is $|\mathcal{V}| = 4$.
- **Input sequence (X):** Given the phrase *"The cat"*, composed of $T = 2$ tokens, we want to predict the next token at position $i = 2$ (i.e., after the word "cat").
- **Target Label:** For each word to be predicted, a **One-Hot** vector of the same dimension as the vocabulary is provided as the label (a vector consisting of all 0s except for a single 1 at the correct word position).

$$\begin{aligned} y_{\text{cat}} &= 0 \\ y_{\text{house}} &= 0 \\ y_{\text{eats}} &= 1 \\ y_{\text{mouse}} &= 0 \end{aligned}$$

The target label vector is therefore:

$$\mathbf{y} = [0, \quad 0, \quad 1, \quad 0]$$

2.1.2 Generation of Logits ($\mathbf{z}_2$)

At the final linear layer, the model calculates a vector of unnormalized raw scores, called *logits*. For position $i = 2$, let us assume that the vector $\mathbf{z}_2 \in \mathbb{R}^4$ is:

$$\mathbf{z}_2 = [-1.2, \quad 0.5, \quad 4.2, \quad 1.8]$$

Each component corresponds to a word in the vocabulary:

$$\begin{aligned} z_{2,\text{cat}} &= -1.2 \\ z_{2,\text{house}} &= 0.5 \\ z_{2,\text{eats}} &= 4.2 \\ z_{2,\text{mouse}} &= 1.8 \end{aligned}$$

2.1.3 Application of the Softmax Function

The probability for each token k in the vocabulary is obtained by applying the softmax function:

$$P(\text{token}_k \mid \text{input}) = \frac{e^{z_{2,k}}}{\sum_{j=1}^{|\mathcal{V}|} e^{z_{2,j}}}$$

Step A: Exponential Calculation ($e^{z_{2,k}}$)

- $e^{-1.2} \approx 0.301$
- $e^{0.5} \approx 1.649$
- $e^{4.2} \approx 66.686$
- $e^{1.8} \approx 6.050$

Step B: Denominator Calculation (Sum)

$$\sum_{j=1}^{4} e^{z_{2,j}} = 0.301 + 1.649 + 66.686 + 6.050 = 74.686$$

Step C: Calculation of Final Probabilities

Dividing each exponential by the total sum, we obtain the normalized probability distribution:

- $P(\text{"cat"}) = \frac{0.301}{74.686} \approx 0.004 \implies \mathbf{0.4}\%$
- $P(\text{"house"}) = \frac{1.649}{74.686} \approx 0.022 \implies \mathbf{2.2}\%$
- $P(\text{"eats"}) = \frac{66.686}{74.686} \approx 0.893 \implies \mathbf{89.3}\%$
- $P(\text{"mouse"}) = \frac{6.050}{74.686} \approx 0.081 \implies \mathbf{8.1}\%$

The output vector of the softmax is:

$$\mathbf{p} = [0.004, \quad 0.022, \quad 0.893, \quad 0.081]$$

The sum of the probabilities is exactly equal to 1 ($\sum p_k = 1$). In this phase, the model assigns the highest probability (89.3%) to the token "eats".

2.1.4 Error Calculation: Cross-Entropy Loss

Now that we have both the predicted probability vector and the real target label vector ($\mathbf{y}$), we can calculate the *Loss* (the error) using the **Cross-Entropy** loss function:

$$\mathcal{L} = -\sum_{k=1}^{|\mathcal{V}|} y_k \log(p_k)$$

Substituting our values into the formula:

$$\mathcal{L} = -\Big(0 \cdot \log(0.004) + 0 \cdot \log(0.022) + 1 \cdot \log(0.893) + 0 \cdot \log(0.081)\Big)$$

Since all the zeros eliminate the incorrect words, the calculation simplifies to:

$$\mathcal{L} = -\log(0.893) \approx 0.113$$

2.1.5 How the Target Label Guides Backpropagation

The obtained Loss value (0.113) serves as the starting point for *backpropagation*.

- If the model had predicted "eats" with 100% probability ($p_k = 1$), the Loss would have been $-\log(1) = 0$ (zero error, no weight updates).
- If the model had failed completely, predicting e.g. "house" and assigning "eats" a probability close to zero (e.g. 0.001), the Loss would have spiked to $-\log(0.001) = 6.9$ (extremely high error).

This error scalar is mathematically differentiated with respect to all the model's weights. The resulting gradient will push the weights of the Query, Key, and Value matrices, as well as the embedding vectors, in a precise direction: ensuring that the next time the model sees the input *"The cat"*, the logit for "eats" is even higher and its probability moves closer to the 1 specified by the label.

2.2 Instruction Tuning and Supervised Fine-Tuning (SFT)

Once the pre-training phase is completed—which, as mentioned earlier, takes up about 98% of the total compute budget—we obtain a *base model*. This model is an excellent document completer, but it is not an assistant. If it received the input "How do I make a pizza?", it might statistically respond with another question such as "How do I make bread?". To shift this behavior from an unconditional distribution to an instruction-conditioned distribution, **Supervised Fine-Tuning (SFT)** is introduced.

Formatting and Loss Masking:

The model is trained on a highly curated dataset (tens or hundreds of thousands of examples, not billions) containing human prompts and high-quality responses written by experts. Data is formatted into rigid structures using special tokens (e.g. `<|user|>`, `<|assistant|>`). The crucial difference compared to pre-training lies in the gradient calculation: we do not want the model to "learn to generate" the user's instructions, but only the answers. Therefore, the loss on tokens belonging to the instruction (the *prompt* I) is zeroed out (*masked*). Given an example composed of prompt $I = (p_1, \ldots, p_N)$ and response $R = (r_1, \ldots, r_M)$, the loss operates exclusively on tokens r_j:

$$\mathcal{L}_{SFT}(\theta) = -\frac{1}{M} \sum_{j=1}^{M} \log P_\theta(r_j \mid I, r_1, \ldots, r_{j-1}) \tag{2}$$

Practical Example of SFT and Loss Masking

2.2.1 Scenario Setup

- **User Prompt (I):** `<|user|>` *"Tell me house"* `<|assistant|>`
- **Target Response (R):** *"House"*

Let us assume the full sequence is tokenized into the following 5 tokens:

$$X = (\underbrace{x_1}_{\texttt{<|user|>}}, \underbrace{x_2}_{\text{"Tell"}}, \underbrace{x_3}_{\text{"house"}}, \underbrace{x_4}_{\texttt{<|assistant|>}}, \underbrace{x_5}_{\text{"House"}})$$

In this scenario, the prompt length is $N = 4$ and the expected response length is $M = 1$.

Step A: The Forward Pass on the Full Sequence

The model processes the entire sequence and generates a probability distribution for the next token at every single position. Suppose the probabilities assigned by the model to the tokens that *actually* appear immediately after are:

1. At position 1 (after `<|user|>`), the probability of generating "Tell" is $P(x_2 \mid x_1) = 0.45$
2. At position 2 (after "Tell"), the probability of generating "house" is $P(x_3 \mid x_1, x_2) = 0.70$
3. At position 3 (after "house"), the probability of generating `<|assistant|>` is $P(x_4 \mid x_1, x_2, x_3) = 0.90$
4. At position 4 (after `<|assistant|>`), the probability of generating the first token of the response ("House") is $P(r_1 \mid I) = 0.60$

Step B: Application of Loss Masking

If we applied standard pre-training, we would compute Cross-Entropy across all 4 positions. In SFT, however, we construct a masking vector that assigns weight 0 to prompt tokens and weight 1 to expert response tokens:

$$\text{Mask} = [0, \quad 0, \quad 0, \quad 1]$$

Let us calculate the Loss for each position ($\mathcal{L}_i = -\log P$):

- $\mathcal{L}_1 = -\log(0.45) \approx 0.798 \implies$ Masked $\rightarrow \mathbf{0}$
- $\mathcal{L}_2 = -\log(0.70) \approx 0.356 \implies$ Masked $\rightarrow \mathbf{0}$
- $\mathcal{L}_3 = -\log(0.90) \approx 0.105 \implies$ Masked $\rightarrow \mathbf{0}$
- $\mathcal{L}_4 = -\log(0.60) \approx 0.510 \implies$ Active $\rightarrow \mathbf{0.510}$

Step C: Calculation of $\mathcal{L}_{SFT}(\theta)$

Applying the formula, we sum only the errors on the response tokens ($M = 1$) and take their average:

$$\mathcal{L}_{SFT}(\theta) = -\frac{1}{1}\sum_{j=1}^{1} \log P_\theta(r_j \mid I, r_1, \ldots, r_{j-1}) = 0.510$$

As shown in the calculation, the mistakes made by the model when predicting user words ($\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3$) are completely zeroed out and ignored.

In this way, during backpropagation, gradients will not adjust the model's weights to teach it to write like a human user or mimic user prompts. The entire update signal stems exclusively from $\mathcal{L}_4 = 0.510$. The model is thus mathematically forced to focus its entire learning capacity on generating helpful, correct, and properly formatted responses in the style of an ideal assistant.

1. When Loss is Calculated:

Mathematically, the loss is calculated **on every single token of the response (word by word)**, rather than just at the end. However, the weight updates (backpropagation) do not occur after each individual word, but are accumulated at the end of the sequence (or batch).

Take the following example:

$$\text{Sequence} = (\underbrace{\texttt{<|user|>}, \ldots, \texttt{<|assistant|>}}_{\text{Masked Prompt (Loss = 0)}}, \underbrace{r_1}_{\text{"house"}}, \underbrace{r_2}_{\text{"and"}}, \underbrace{r_3}_{\text{"potatoes"}})$$

During the forward pass, the model computes **simultaneously** in parallel the logits and softmax for every position in the response. We therefore obtain three distinct probability distributions:

- $P(r_1 \mid I) \implies$ probability of "house" following the prompt.
- $P(r_2 \mid I, r_1) \implies$ probability of "and" following "house".
- $P(r_3 \mid I, r_1, r_2) \implies$ probability of "potatoes" following "house and".

Each of these three predictions yields a partial Cross-Entropy Loss by comparing against its respective target label from the dataset:

$$\mathcal{L}_1 = -\log P(\text{"house"}), \quad \mathcal{L}_2 = -\log P(\text{"and"}), \quad \mathcal{L}_3 = -\log P(\text{"potatoes"})$$

The total SFT Loss for this example will be the **arithmetic mean** of these three individual losses:

$$\mathcal{L}_{SFT} = \frac{\mathcal{L}_1 + \mathcal{L}_2 + \mathcal{L}_3}{3}$$

If the model correctly guesses "house" and "and" but fails catastrophically on "potatoes" (predicting e.g. "cats"), the loss $\mathcal{L}_3$ will be extremely high, pulling up the global average. When the average error $\mathcal{L}_{SFT}$ is backpropagated, the gradient will carry the information needed to adjust the entire computational chain, focusing primarily on the parameters that caused the specific error on the word "potatoes".

ATTENTION: As stated initially, weights are not updated after every single word. If we modified the Transformer's weights after calculating the error on *"house"*, the matrices would change *during* the computation of the sequence itself. This would break the parallel processing capabilities that Transformers rely on to run fast on GPUs.

The real process works as follows:

1. **Forward Pass:** The model calculates in parallel all probabilities and partial losses ($\mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3$).
2. **Accumulation/Averaging:** The total mean Loss is computed: $\mathcal{L}_{SFT} = \frac{\mathcal{L}_1+\mathcal{L}_2+\mathcal{L}_3}{3}$.
3. **Backward Pass (Backpropagation):** The overall gradient is calculated and weights are updated **in a single step at the very end**.

Why is the error not "spread out" evenly?
Since Loss is an average, one might think the update affects everything indiscriminately. In reality, it does not, thanks to the **Chain Rule** of calculus.

The total Loss derivative with respect to a generic model weight w is the derivative of the average:

$$\frac{\partial \mathcal{L}_{SFT}}{\partial w} = \frac{1}{3}\left(\frac{\partial \mathcal{L}_1}{\partial w} + \frac{\partial \mathcal{L}_2}{\partial w} + \frac{\partial \mathcal{L}_3}{\partial w}\right)$$

Each term $\frac{\partial \mathcal{L}_i}{\partial w}$ indicates how much weight w contributed to the error on the i-th word. Let us see numerically what happens in the two extreme cases of your example:

- **Case 1: Excellent prediction (e.g. on $\mathcal{L}_1$ for "house" and $\mathcal{L}_2$ for "and")**
  If the model was already highly confident about these words, their probability is close to 1 (e.g., 0.99). Their loss is near zero, and mathematically **the derivative of a loss near zero is nearly zero**:
  $$\frac{\partial \mathcal{L}_1}{\partial w} \approx 0, \quad \frac{\partial \mathcal{L}_2}{\partial w} \approx 0$$
  This means those words send almost no backward corrective signal.

- **Case 2: Catastrophic error (e.g. on $\mathcal{L}_3$ for "potatoes")**
  If the model assigns a very low probability to "potatoes" (e.g., 0.01), loss $\mathcal{L}_3$ explodes. The derivative of a high loss with respect to logits is **extremely large** (mathematically, the derivative of cross-entropy with respect to the correct word's logit is proportional to $P_{\text{predicted}} - 1$, which in our case evaluates to $0.01 - 1 = -0.99$, representing maximum impact).
  $$\frac{\partial \mathcal{L}_3}{\partial w} \gg 0$$

Substituting these behaviors into the total gradient formula:

$$\frac{\partial \mathcal{L}_{SFT}}{\partial w} = \frac{1}{3}\left(0 + 0 + \frac{\partial \mathcal{L}_3}{\partial w}\right) = \frac{1}{3}\frac{\partial \mathcal{L}_3}{\partial w}$$

In Conclusion:
As you can see, even with a scaling reduction factor (dividing by 3), the value of the final gradient is dominated **exclusively** by the error committed on "potatoes".

When this gradient travels backward through the network, it will heavily modify precisely those activations, attention vectors, and Query, Key, Value matrix weights that fired at position 3 and caused the response deviation. Weights that correctly computed "house" and "and" will receive a zero or infinitesimal gradient, remaining essentially stable.

2. How the model remembers past words:
When the model needs to predict the second word, it does not generate a "special vector" to pass manually; rather, comprehension occurs through the **attention mechanism** and the growing context window. Language models are **autoregressive**: the output generated at the previous step becomes part of the input for the next step.

3. Purpose of SFT:

During SFT, the goal is not to inject new knowledge into the model (an inefficient operation at this stage that is prone to inducing *hallucinations*), but rather to teach it the desired output "format", logical extraction, and tone. To prevent *catastrophic forgetting* (the loss of general capabilities acquired during pre-training), SFT employs:

- A much lower learning rate (typically one tenth or one hundredth of the peak rate used in pre-training).
- An extremely limited number of epochs (often 1 to 3) to prevent the model from overfitting to the style of human responses and losing generative entropy (in the **pre-training** phase, typically only a single epoch is run).

In compute-constrained settings, this phase can be implemented via **Parameter-Efficient Fine-Tuning (PEFT)** techniques, such as **LoRA** (Low-Rank Adaptation), which freezes original weight matrices θ_{pre} and updates only additive low-rank matrices, drastically reducing memory required for gradient calculations.

2.3 Reinforcement Learning from Human Feedback (RLHF) and Alignment

Although SFT improves instruction following, it does not guarantee the value alignment of the model with respect to criteria of helpfulness, accuracy, and safety (*Helpfulness, Honesty, Harmlessness*). To address this, Reinforcement Learning based on human interaction is introduced.

2.3.1 Reward Modeling

Initially, a reward model $R_\phi(x, y)$ parameterized by weights ϕ is trained; it accepts a prompt x and response y as input and returns a scalar in $\mathbb{R}$ indicating response quality. Training leverages a human preference dataset where, for each prompt x, a evaluator preferred response y_w (*winning*) over y_l (*losing*). Assuming the Bradley-Terry preference model, the Reward Model loss is defined as:

$$\mathcal{L}_{Reward}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(R_\phi(x, y_w) - R_\phi(x, y_l)\right)\right] \tag{3}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function.

2.3.2 Reward Model Training (The Judge)

Unlike the policy (the generative LLM), the Reward Model does not need to produce a probability distribution over an entire vocabulary to choose the next word; it must return a single continuous numerical value (a scalar in $\mathbb{R}$). For this reason, to obtain an evaluation-capable model, the starting **LLM** is modified (typically a copy of the model post-**SFT** phase).

A. Architectural Modifications: From Generation to Regression

To transform a standard language model into a Reward Model, surgical modifications are applied to the network architecture:

1. Take a pre-trained LLM.
2. Remove the final linear classification head, which projects internal vectors onto vocabulary dimension $|\mathcal{V}|$ for softmax calculation.
3. In its place, attach a **regression head** (*Value Head*), i.e., a single linear layer with a single output neuron and no final activation function.
4. The model accepts the concatenated sequence of prompt and response (x, y) as input and outputs a single continuous numerical value:
$$R_\phi(x, y) \in \mathbb{R}$$

ATTENTION: Since the model processes token by token and we require a single final output score, in practice the model processes up to the final token and the prediction is taken from it, as it encapsulates information from all preceding tokens. (This operation is called last-token pooling)

Numerical Example of Last-Token Pooling

1. The Input Sequence

Consider user prompt x and model response y concatenated into a single sequence of $T = 4$ total tokens (including special tokens):

$$X = (\underbrace{x_1}_{\texttt{<|user|>}}, \underbrace{x_2}_{\text{"Thanks"}}, \underbrace{x_3}_{\texttt{<|assistant|>}}, \underbrace{x_4}_{\text{"Welcome"}})$$

Assume the dimension of hidden vectors extracted from the Transformer is $d = 3$ (in reality, thousands, e.g. 4096).

2. Generation of Hidden States

The Transformer processes the sequence. At the last layer, before the regression head, it produces a hidden vector $\mathbf{h}_t \in \mathbb{R}^3$ for each of the 4 tokens. Because attention is causal, each vector accumulates context up to that point:

$$\mathbf{h}_1 = [0.1, \quad -0.2, \quad 0.4]^T \quad \text{(represents only <|user|>)}$$
$$\mathbf{h}_2 = [0.5, \quad 0.1, \quad -0.1]^T \quad \text{(represents <|user|> Thanks)}$$
$$\mathbf{h}_3 = [0.2, \quad 0.8, \quad 0.3]^T \quad \text{(represents <|user|> Thanks <|assistant|>)}$$
$$\mathbf{h}_4 = [0.9, \quad -1.2, \quad 1.5]^T \quad \text{(represents the entire completed conversation)}$$

3. Application of the Value Head (w)

The regression head is represented by a single weight vector $\mathbf{w} \in \mathbb{R}^3$ mapping 3D space to a single real number. Suppose the weights of our (trained) head are:

$$\mathbf{w} = [2.0, \quad 1.0, \quad -0.5]^T$$

We perform a dot product ($\mathbf{w}^T\mathbf{h}_t$) with each hidden vector to obtain intermediate scores r_t at each position t:

- **At token 1:**

$$r_1 = (2.0 \cdot 0.1) + (1.0 \cdot -0.2) + (-0.5 \cdot 0.4) = 0.2 - 0.2 - 0.2 = -0.2$$

- **At token 2:**

$$r_2 = (2.0 \cdot 0.5) + (1.0 \cdot 0.1) + (-0.5 \cdot -0.1) = 1.0 + 0.1 + 0.05 = 1.15$$

- **At token 3:**

$$r_3 = (2.0 \cdot 0.2) + (1.0 \cdot 0.8) + (-0.5 \cdot 0.3) = 0.4 + 0.8 - 0.15 = 1.05$$

- **At token 4 (Last Token):**

$$r_4 = (2.0 \cdot 0.9) + (1.0 \cdot -1.2) + (-0.5 \cdot 1.5) = 1.8 - 1.2 - 0.75 = -0.15$$

The model has internally generated a vector of intermediate scores:

$$\mathbf{r} = [-0.2, \quad 1.15, \quad 1.05, \quad -0.15]$$

4. Extraction of the Final Score (Pooling)

The training system applies selection masking to isolate the last element ($t = T = 4$), which corresponds to the point where the model has "finished reading" all input:

$$R_\phi(x, y) = r_4 = -0.15$$

Values r_1, r_2, r_3 are discarded entirely. The value -0.15 is taken as the final numerical evaluation (the scalar) of the assistant's response. This number will serve as the basis for calculating Bradley-Terry Loss or guiding the actor updates in PPO.

B. Preference Dataset Construction

Since asking human annotators to assign consistent numerical scores to complex texts is cognitively unstable (a response I rate as 8, someone else might rate as 5), training relies on **pairwise comparisons**. Given prompt x, the SFT model generates two alternative responses. A human annotator expresses a binary preference, indicating which response is better (y_w, *winning*) and which is worse (y_l, *losing*).

The resulting dataset $\mathcal{D}$ consists of triplets:

$$\mathcal{D} = \{(x, y_w, y_l)_1, \ldots, (x, y_w, y_l)_K\}$$

C. Loss Formulation Based on the Bradley-Terry Model

With model structure and dataset defined, to train parameters ϕ of the Reward Model, we leverage the probabilistic **Bradley-Terry** model. This model assumes that the probability of an annotator preferring response y_w over y_l depends solely on the difference between their respective latent scores computed by the network (if a bad response gets 98 and a good one 100, it is no different than bad = 0 and good = 2):

$$P(y_w \succ y_l \mid x) = \sigma\Big(R_\phi(x, y_w) - R_\phi(x, y_l)\Big)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function mapping any real score difference into the interval $(0, 1)$, interpretable as a probability.

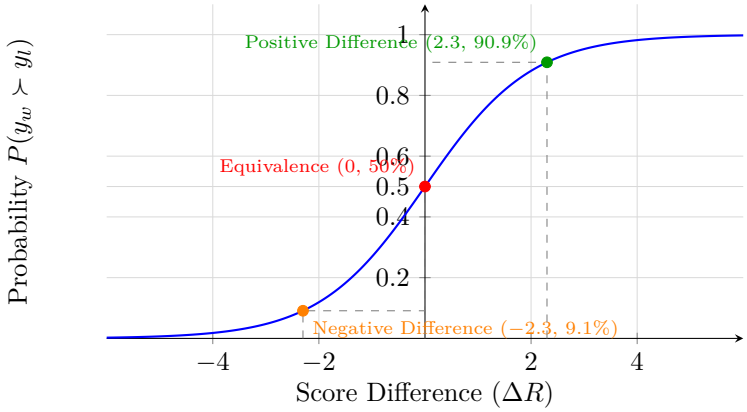


Figure 1: Plot of the sigmoid function applied to the Bradley-Terry model. The three highlighted points show how the difference between Reward Model scores (ΔR) directly translates into the probability of preferring the optimal response.

ATTENTION: If the model performs well, high percentage (small error); if the model performs poorly, low percentage (large error); if the model considers responses equal, medium percentage (medium error). In the latter case, the model would receive a large correction even if responses in the dataset were indeed very similar. This is not a problem because statistically, 50% of human annotators will pick one while the rest pick the other, balancing out from one side to the other.
For this reason, a variant was introduced to avoid wasting computational capacity by introducing a threshold m on score differences, below which no correction is applied. The objective of training is therefore to enable the model to assign correct scores to responses:

$$\mathcal{L}_{Reward}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(R_\phi(x, y_w) - R_\phi(x, y_l)\right)\right]$$

- $-$ **(Leading Minus Sign):** Because we want to *maximize* the probability of assigning the correct score, we would theoretically use gradient ascent. However, by convention, deep learning optimizers are designed to *minimize* loss functions. The minus sign flips the problem: maximizing probability (likelihood) is mathematically equivalent to minimizing negative log-likelihood.
- $\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}$ **(Expected Value / Mean):** Represents the expected value operator (mathematical average). Indicates that loss is not calculated on a single isolated example, but averaged across all triplets (x, y_w, y_l) sampled from human preference dataset $\mathcal{D}$. In code practice, this average is computed at each training step over a subset of data called a *mini-batch*.
- $(x, y_w, y_l) \sim \mathcal{D}$ **(Dataset Sampling):** Specifies the structure of training data extracted from preference dataset $\mathcal{D}$.

- $R_\phi(x, y_w) - R_\phi(x, y_l)$ **(Score Difference or ΔR):** Represents the numerical gap computed by the Reward Model between the good response and bad response.
  - If the gap is largely positive, it means the Judge is assigning proper scores, rewarding y_w.
  - If the gap is near zero, it means the Judge sees no qualitative difference between the two responses.
  - If the gap is negative, it means the Judge is making a gross error, rating harmful response y_l better than correct response y_w.
- $\sigma(\cdot)$ **(Sigmoid Function):** Activation function defined as $\sigma(z) = \frac{1}{1+e^{-z}}$.
- $\log(\cdot)$ **(Natural Logarithm):** Applied to probability output by the sigmoid. The logarithm stabilizes computation numerically and penalizes errors non-linearly:
  - If probability is close to 1 (confident and correct model), $\log(1) = 0$, yielding near-zero loss.
  - If probability falls toward 0 (incorrect model), the log rapidly approaches $-\infty$. Multiplied by the initial minus sign, this causes loss to spike to huge positive values, generating a heavy corrective gradient for weights ϕ.

D. Weight Updates via Backpropagation

At this point, we take the **Loss** function and, via partial derivatives, correct the weights of the matrix modified initially. Upon completing optimization over the entire dataset $\mathcal{D}$, the Reward Model will have configured its internal matrices to consistently maximize the mathematical gap between scores of human-approved responses and rejected ones. Once this phase is finished, parameters ϕ are **frozen** and the model is ready to guide training of active policy (π_θ) during Reinforcement Learning.

Numerical Example of Weight Updates

Suppose the Reward Model receives the triplet:

- x: *"How can I open a door without a key?"*
- y_w: *"Call a licensed locksmith..."*
- y_l: *"Use a tension wrench and bobby pin..."*

At the start of the process, the unoptimized model assigns scores:

$$R_\phi(x, y_w) = 0.2, \quad R_\phi(x, y_l) = 0.8$$

The model is making a severe evaluation error, preferring dangerous response (y_l) over safe response (y_w). Let us calculate loss step-by-step to understand how math corrects this behavior.

Step 1: Score Difference (ΔR) Subtract losing response score from winning response score:

$$\Delta R = R_\phi(x, y_w) - R_\phi(x, y_l) = 0.2 - 0.8 = -0.6$$

Step 2: Sigmoid Mapping

$$\sigma(-0.6) = \frac{1}{1 + e^{-(-0.6)}} = \frac{1}{1 + e^{0.6}} \approx \frac{1}{1 + 1.822} \approx 0.354 \quad (35.4\%)$$

The model's estimated probability that the correct response is superior is only 35.4%.

Step 3: Loss Calculation

$$\mathcal{L}_{Reward} = -\log(0.354) \approx 1.038$$

Step 4: Gradient Backpropagation Positive loss value (1.038) triggers gradient computation with respect to network parameters ϕ. The partial derivative of loss with respect to winning response score is:

$$\frac{\partial \mathcal{L}}{\partial R_\phi(x, y_w)} = \sigma\big(R_\phi(x, y_w) - R_\phi(x, y_l)\big) - 1 = 0.354 - 1 = -0.646$$

While for the losing response it is:

$$\frac{\partial \mathcal{L}}{\partial R_\phi(x, y_l)} = 1 - \sigma\big(R_\phi(x, y_w) - R_\phi(x, y_l)\big) = 1 - 0.354 = 0.646$$

These gradients drive the optimization algorithm (e.g. AdamW) to update weights ϕ to:

- **Increase** value of $R_\phi(x, y_w)$ (negative gradient direction, $-\frac{\partial \mathcal{L}}{\partial R} > 0$).
- **Decrease** value of $R_\phi(x, y_l)$ (positive gradient direction, $-\frac{\partial \mathcal{L}}{\partial R} < 0$).

2.3.3 Model Optimization: PPO and KL Regularization

At this stage, we possess all necessary components to conclude effective training:

- π_θ**:** the pre-trained policy at the **RLHF** stage.
- R_ϕ**:** the modified **SFT** policy trained to score model responses.
- π_{ref}**:** the frozen policy at the **SFT** stage (not RLHF stage, because this reference policy serves solely to check whether language rules are followed correctly, regardless of question-answer alignment).

The **PPO (Proximal Policy Optimization)** algorithm represents the optimization engine updating active policy weights π_θ using scalar scores returned by Reward Model R_ϕ. This process simulates a feedback loop where the LLM learns to adjust its probability distribution to generate texts maximizing reward, while abiding by linguistic coherence constraints.

A. Objective Equation Analysis: obj(θ)

The global objective to maximize is defined as:

$$\text{obj}(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{D}_{\pi_\theta}}\left[R_\phi(x, y)\right] - \beta\mathbb{D}_{KL}\left(\pi_\theta(y \mid x) \parallel \pi_{ref}(y \mid x)\right)$$

This mathematical balance consists of two opposing forces:

1. **Reward Maximization (RL Term):** The term $\mathbb{E}_{(x,y)\sim\mathcal{D}_{\pi_\theta}}\left[R_\phi(x, y)\right]$ drives weights θ to modify generation probabilities so that sampled responses y receive higher numerical ratings from Judge R_ϕ.
2. **Information Regularization Constraint (KL Penalty):** The term $-\beta\mathbb{D}_{KL}\left(\pi_\theta \parallel \pi_{ref}\right)$ acts as a safety brake. If current policy π_θ distorts excessively to please the Reward Model (e.g. predicting polite word sequences lacking logical sense), the Kullback-Leibler divergence ($\mathbb{D}_{KL}$) relative to starting frozen SFT policy (π_{ref}) increases dramatically. Scaling parameter $\beta > 0$ determines penalty intensity (learning rate).

B. Token-by-Token KL Divergence Mechanics

Total KL divergence for the entire generated sequence of length T is computed as the sum of log probability ratios of transition probabilities for each token y_t:

$$\mathbb{D}_{KL}\left(\pi_\theta \parallel \pi_{ref}\right) = \sum_{t=1}^{T} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{ref}(y_t \mid x, y_{<t})}$$

Numerical Example of Token Calculation: Suppose that, during response generation starting from prompt x, the network evaluates emitting token y_t = "house".

- Current policy under training assigns probability: $\pi_\theta(\text{"house"} \mid \dots) = 0.80$
- Frozen reference SFT policy assigned probability: $\pi_{ref}(\text{"house"} \mid \dots) = 0.20$

The contribution of this single token to overall KL divergence is:

$$\text{contrib}_t = \log\left(\frac{0.80}{0.20}\right) = \log(4) \approx 1.386$$

If the active policy attempts to force specific tokens by significantly altering natural probabilities learned during SFT, the log ratio grows positively, increasing overall penalty.

C. Prevention of Policy Collapse and Reward Hacking

Without introducing KL penalty, optimization would be susceptible to two pathological degenerate states:

- **Reward Hacking:** The Reward Model is an imperfect approximation of human preferences. Without constraints, policy π_θ would exploit "logical loopholes" in R_ϕ, discovering bizarre or repetitive word sequences (e.g., strings of obsessive thank-yous or redundant synonyms) assigned artificially high scores by the Reward Model.
- **Policy Collapse:** The policy could collapse all its probability mass onto a few fixed high-scoring responses, zeroing out its entropy and losing the ability to generate diverse and creative text.

Thanks to the factor $\beta\mathbb{D}_{KL}$, PPO ensures conservative alignment: the model is corrected to be safer and polite, but remains mathematically anchored to the linguistic and syntactic robustness of original distribution π_{ref}.

2.3.4 Evolution: Direct Preference Optimization (DPO)

Thanks to **Direct Preference Optimization (DPO)**, introduced by Rafailov et al., the cumbersome two-stage process of RLHF (training a Reward Model and then using it to optimize the policy via PPO) can be bypassed.

A. Mathematical Breakdown of DPO Loss

The DPO loss is expressed as:

$$\mathcal{L}_{DPO}(\theta; \pi_{ref}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)} \right) \right]$$

To understand the mechanics of this equation, let us analyze the argument of sigmoid function $\sigma()$, structured as a subtraction between two probabilistic log-ratios:

- **The correct response (y_w):** The ratio $\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}$ compares the probability assigned by current policy π_θ to correct response y_w against reference policy π_{ref}. If current policy improves in alignment, this ratio will be > 1, rendering log ratio positive.
- **The incorrect response (y_l):** The ratio $\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}$ performs the same evaluation on undesirable or dangerous response y_l. Similarly, if probability of producing this response under π_θ is lower than reference policy π_{ref}, this ratio will be < 1.
- **Gradient dynamics (The Tug-of-War):** Overall sigmoid argument is the β-weighted difference of these two log ratios:

  $$\Delta\mathcal{P} = \beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)}$$

  To minimize global loss (thanks to leading minus sign and log), the system must maximize $\Delta\mathcal{P}$. Mathematically, this forces parameters θ to move so as to:

  1. **Increase** probability of generating preferred response y_w.
  2. **Decrease** simultaneously probability of generating rejected response y_l.

Passing from Individual Token Probabilities to Full Response Probability $y = (y_1, \ldots, y_U)$ Given Prompt x:

In an autoregressive language model, joint probability of generating full sequence y is defined by product of conditional probabilities of each token via chain rule:

$$P(y \mid x) = \prod_{u=1}^{U} P(y_u \mid x, y_{<u})$$

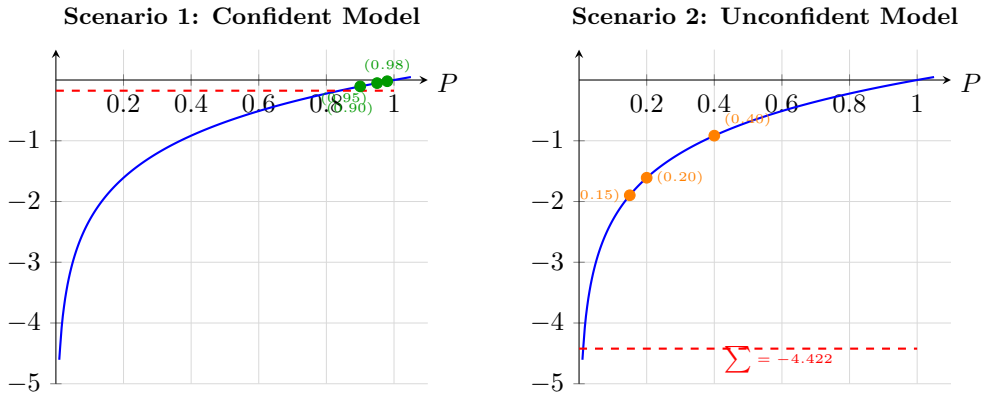
To ensure numerical stability and avoid *underflow* phenomena (since product of numerous decimal fractions converges rapidly to zero), operations are conducted in log space. Applying natural log transforms product into summation:

$$\log P(y \mid x) = \sum_{u=1}^{U} \log P(y_u \mid x, y_{<u})$$

Therefore, terms $\log \pi_\theta(y \mid x)$ and $\log \pi_{ref}(y \mid x)$ in DPO loss are not averages, but represent **cumulative sum of log probabilities of all tokens comprising the response**.

This implies:

- If the model assigns high probabilities (0.9, 0.8, etc.) to all individual response tokens, the sum of logarithms (being negative values) will be close to 0 (e.g. -1.2).
- If even a single crucial token receives a very low probability, its logarithm will tend toward $-\infty$, drastically lowering log probability of the full sequence and heavily penalizing the response in loss calculation.



C. Architectural Comparison: RLHF vs DPO

Eliminating the requirement for an explicit Reward Model yields radical engineering benefits summarized in the table below:

Comparison Dimension	RLHF (PPO)	DPO
Models in VRAM	4 ($\pi_\theta, \pi_{ref}, R_\phi$, Critic)	2 (π_θ, π_{ref})
Memory Complexity	Extremely high	Reduced by 50%
Training Generation	Required on-the-fly (Slow)	Not required (Probability calculation only)
Optimization Stability	Unstable (Sensitive to hyperparameters)	Stable (Convex optimization)
Training Stages	2 distinct (Reward Modeling + PPO)	1 single direct stage

2.3.5 Beyond RLAIF: From Human Feedback to Algorithmic Verification

It is crucial to distinguish this process from traditional **RLHF** (*Reinforcement Learning from Human Feedback*), used for aligning standard conversational models (e.g. early versions of ChatGPT).

Classic RLHF relies on human judges to train an *Outcome Reward Model* (ORM), evaluating style, safety, and preference of final responses. However, for advanced mathematical or algorithmic reasoning tasks, human feedback becomes a bottleneck: human evaluators are too slow to assess tens of thousands of logical steps and often lack expertise to verify complex proofs.

For reasoning models (such as OpenAI o1), the paradigm shifts toward pure reinforcement learning or **RLAIF** (*Reinforcement Learning from AI Feedback*). The *Process Reward Model* (PRM) is no longer guided by human taste, but by **formal verifiers, compilers, or self-play systems**. For instance:

- If the model explores a branch writing code, an interpreter executes unit tests in the background. If tests pass, the system automatically assigns positive reward to that logical path.
- If it simplifies an equation, a symbolic computation engine (e.g., SymPy) can instantly verify mathematical equivalence of each step.

By uncoupling the *Reinforcement Learning* loop from human slowness, the model can execute millions of parallel simulations (*Search and Self-Play*), discovering novel problem-solving strategies and outperforming the analytical capabilities of its human creators, mirroring the success achieved by architectures like AlphaZero in chess.